SIMATS ENGINEERING
CSA1023 – SOFTWARE ENGINEERING
ASSIGNMENT 2

CO2: Analyze software requirements to design scalable and efficient system architectures using collaborative tools like Jira and Confluence, along with structured modeling techniques such as UML diagrams and architectural design patterns.

Student Name	B. Charan Kumar Reddy	**Register Number**	192472195
Department / Section	BE-CSE-AI	Academic Year / Semester	2025-2026
Faculty Name	V. Karpagam	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission	05/11/2026	Date of Evaluation	

MARK SHEET (Rubric-based Evaluation)

S. No	**Task**	**Max**	**Marks Awarded / Remarks**
1	Q1 – Stakeholder investigation and conflict resolution	5	
2	Q2 – Prioritized and testable requirements specification	8	
3	Q3 – UML modeling and requirements traceability	7	
4	Q4 – Alternative architectures and justified design	8	
5	Q5 – Critical-behaviour validation and design revision	7	
6	Q6 – Design Challenge: redesign for expansion and partial failure	10	
7	Q7 – Individual technical reflection	5	
	TOTAL	50	

OVERALL REMARKS:

Student Signature Faculty Signature Program Director

CSA10 – SOFTWARE ENGINEERING
ASSIGNMENT 2

Assignment Title: Design and Validate Requirements and Architecture for the Omnichannel Order Fulfilment Platform

Nature of Assignment: Individual **Duration:** One week **Total Marks:** 50

Scenario — Order Fulfilment Requirements and Scalable Architecture

A retailer needs an order fulfilment platform that connects checkout, stock reservation, payment callbacks, warehouse picking, courier tracking and refunds without losing consistency or exposing restricted customer information. Repeated payment callbacks, two buyers competing for the last unit and failed courier synchronization create conflicts between speed, availability, accountability and confidentiality. This report investigates stakeholder needs, specifies testable requirements, models the system in UML with traceability, compares two architectures, validates critical behaviour with an executable model, and redesigns the selected architecture for expansion from three to twelve regions.

Labelling convention used throughout this report

- **SIMULATED** – stakeholder responses, workload figures and thresholds are planning assumptions created for this assignment; no real stakeholders, customer data, card data or live integrations were used.
- **ASSUMPTION** – a value I fixed to make a requirement measurable (e.g. GST rate, reservation time-out).
- **DEMONSTRATED** – a result actually produced by running the executable model in Appendix B (observed output reproduced there).
- **PREDICTED / PROPOSED** – expected behaviour or design extensions that were not executed.

Model results show that the stated rules are internally consistent in an in-memory model. They are not proof of production accuracy, performance, live inventory control or secure payment handling.

Q1 – Investigate Stakeholder Needs and Resolve Conflicts

1.1 Investigation questions

1. Which events cause the most disputes today: duplicate charges, overselling, late tracking, or refund delays? (customer, support agent)

2. When two customers want the last unit, who should win, and for how long may stock be held while payment is pending? (customer, picker, store manager)

3. What order and payment information must a support agent see to help a customer, and what must stay hidden? (support agent, administrator)

4. Which refunds need approval, who may approve, and how quickly must a decision be made? (finance reviewer, support agent)

5. What should happen to a picker's scans when the warehouse network or courier API is unavailable? (picker, store manager)

6. Which management reports are really needed, and how fresh must they be? (store manager, finance reviewer)

1.2 Stakeholder findings

Stakeholder (simulated)	Simulated finding	Design implication
Customer	Wants checkout in a few steps; was charged twice once when the payment page was refreshed; cannot see where the parcel is.	Idempotent payment handling; order tracking page with clear status.
Picker	Pick lists show items that are already gone; scanners lose Wi-Fi in the back of the warehouse.	Atomic stock reservation; offline scan buffer with unique event IDs.
Support agent	Needs to answer “where is my order?” and “can I get a refund?” but should not see card or payment details; sometimes asks for approval rights to be quick.	Role-based read model without payment fields; no approval right for support.
Finance reviewer	Refunds are approved in chat; no record of who approved what; wants approval before money leaves.	Approval-gated refund workflow with audit trail.
Store manager	Wants daily sales, stock and refund totals; does not need real-time values.	Consolidated reports from summary data, not live queries on order tables.
Administrator	Needs to grant roles, but wants the server to enforce them, not the screen.	Server-side RBAC; role assignment audited.

1.3 Stakeholder analysis

Stakeholder	Influence	Interest	Primary need / engagement
Customer	Medium	High	Fast, trustworthy checkout; tracking. Engage by usability test.
Picker	Low	High	Accurate pick lists; offline tolerance. Engage by walkthrough.
Support agent	Medium	High	Order visibility without payment data.

Stakeholder	Influence	Interest	Primary need / engagement
Finance reviewer	High	High	Auditable refund approval. Key decision-maker for refund rules.
Store manager	High	Medium	Reports, dispatch deadlines.
Administrator	High	Medium	Role control, configuration.

1.4 Conflicting expectations and resolutions

ID	Conflict	Resolution	Justification
C1	Customer: instant checkout. Picker/manager: stock must be exact (no overselling). Payment gateway may be slow.	Reserve stock atomically at order creation for a limited time (ASSUMPTION: 15 min); confirm payment asynchronously; release on expiry.	Keeps checkout fast (no waiting for gateway) while ensuring the last unit is sold once. Rejected: no reservation (oversell risk) and lock-until-paid (abandoned carts block stock).
C2	Support agent: wants full order visibility to help quickly. Privacy: payment data must be restricted.	Support sees order status, items, shipment and refund status; payment amount/gateway identifiers are hidden by the API, not the UI.	Agents resolve most queries from status; hiding is enforced server-side so UI role selection cannot grant access.
C3	Support/customers: fast refunds. Finance: no refund without approval.	Every refund needs finance approval in Release 1; support may only request; a 4-working-hour approval target is set; high-value refunds need a second approver.	Speed is improved by a queue and target time, not by bypassing control. The rule is configurable per region later (Q6).
C4	Picker: keep scanning when offline. Management: stock must stay consistent.	Offline scans are buffered with unique IDs and applied once on sync; the server remains the authority for stock.	Availability for pickers without letting a local device decide global stock.
C5	Store manager: real-time reports. Cost/performance: reports must not slow checkout.	Reports run on summary views refreshed every 5 minutes (ASSUMPTION).	Meets management needs with limited infrastructure cost.

Stakeholder requests that were **not** turned into requirements: (a) support-agent request for refund approval rights – rejected under C3; (b) real-time dashboards – replaced by 5-minute summaries; (c) live card-data display for agents – rejected for privacy; (d) own payment gateway – out of scope.

1.5 System boundary, actors and exclusions

Item	Definition
Inside the boundary	Catalogue browsing, cart and checkout, stock reservation, payment callback handling, picking, dispatch, courier tracking synchronization, returns and refunds, support order view, reporting, role management, audit log.
Human actors	Customer, Picker, Support Agent, Finance Reviewer, Store Manager, Administrator.
External systems (test stubs, status disclosed)	Payment Gateway, Courier API, Notification (email) service. All are simulated with controlled test interfaces; no live access.

Item	Definition
Exclusions	Real card handling (PCI scope stays with gateway), real customer data, tax filing, multi-currency, mobile native apps, demand forecasting (ML) – labelled as proposed extension only.

Q2 – Prioritized and Testable Requirements Specification

2.1 Tool arrangements examined and selected

Criterion	Arrangement 1: Jira + Confluence + PlantUML	Arrangement 2: GitHub Issues/Projects + Wiki + Mermaid	Arrangement 3: Azure DevOps Boards + Wiki
Traceability	Issue links and Confluence page links; requirement IDs as issue keys (OFP-n).	Issue references and markdown tables; manual links.	Work-item links; good for repos.
Collaboration	Comments, mentions, page history, review workflow.	PR-style review; good for developers.	Strong with pipelines.
Version history	Page versions; issue history.	Git history (strong).	Wiki is Git-backed.
Diagram support	PlantUML text kept as code blocks/attachments; all UML types.	Mermaid rendered natively; fewer UML types.	Mermaid/images.
Accessibility	Keyboard navigation and published accessibility documentation (verify current status).	Good keyboard support.	Good.
Cost	Free tier up to 10 users; paid beyond (verify current pricing).	Free for small teams.	Free for 5 users.

Selected: Jira (requirement records, priorities, links) + Confluence (specification, ADRs) + PlantUML (UML as text so diagrams can be versioned and re-rendered). This reuses the Jira project and workflow skills already applied in Assignment 1 and gives explicit issue links for traceability.

Limitations and mitigations: (1) Jira has no native UML – diagrams are generated from PlantUML text and attached with the source. (2) Cost above 10 users – budget the paid plan only if the team grows. (3) Diagram and requirement drift – every diagram page lists the requirement IDs it covers, and the traceability matrix (Q3) is reviewed at each change. (4) Jira/Confluence screenshots must be captured separately (see Appendix A).

2.2 Planning assumptions and order rules

Rule / assumption	Definition (ASSUMPTION where stated)
Price and tax	Line total = quantity × unit price. Tax = 18% GST (ASSUMPTION) on the order subtotal, rounded to 2 decimals. Order total is fixed at placement; later price changes do not affect it.
Stock reservation	Reserve atomically when the order is placed; available = stock − reserved; reservation expires after 15 minutes without confirmed payment (ASSUMPTION; added after validation S7).
Ownership	A customer may view or cancel only their own orders; an order belongs to exactly one customer.
Permitted status changes	Placed→Paid→Packed→Dispatched→Delivered→Closed; Placed→Expired/Cancelled; Paid→Cancelled (with refund); Delivered→Returned→Closed.
Dispatch deadline	Paid orders must be dispatched within 24 hours (ASSUMPTION); orders still Packed/Paid after 24 h appear on the store manager's overdue list.
Duplicate callbacks	A gateway event ID is processed at most once; repeats return success but cause no state change.
Refund approval	Only a finance reviewer can approve or reject. Refunds above ₹10,000 (ASSUMPTION) need a second approver (store manager). Execution is allowed only from Approved/RetryPending.
Workload (Release 1)	50 simultaneous users; 100 stock, payment and scan events per minute; synthetic data only.

2.3 Functional requirements (14 + 1 added after validation)

ID	Requirement	Source	Priority	Acceptance criteria
FR-01	Customer browses the catalogue and adds items to a cart.	Customer	Must	Given a catalogue of 100 synthetic products, search/filter returns correct items in 10 of 10 test queries.

ID	Requirement	Source	Priority	Acceptance criteria
FR-02	System calculates price and tax at checkout.	Customer, Finance	Must	Total = subtotal + 18% tax (rounded) in 20 of 20 test cases; total unchanged after later price change.
FR-03	System reserves stock atomically when an order is placed.	Picker, Manager	Must	With 1 unit and 2 concurrent buyers, exactly 1 order is created and 1 refused.
FR-04	System accepts payment callbacks and confirms the order once per gateway event ID.	Customer, Finance	Must	Same event ID sent 5 times concurrently → 1 payment, 1 confirmation, stock reduced once.
FR-05	System enforces permitted order status changes only.	Manager, Support	Must	Any illegal transition (e.g. Placed→Dispatched) is rejected and logged in 100% of 15 transition tests.
FR-06	Customer tracks order status and shipment.	Customer	Must	Tracking page shows the current status within 5 s of a status change.
FR-07	Picker sees a pick list and confirms picking.	Picker	Must	Only orders in Paid appear; confirm moves to Packed; non-pickers denied.
FR-08	Dispatch only after confirmed payment and pick confirmation.	Manager	Must	Dispatch attempt on unpaid or unpacked order is rejected in 100% of cases.
FR-09	Courier shipment creation is retried safely if the courier API fails.	Engineer, Support	Must	After 2 simulated failures the shipment is created once with no loss of the stored order.
FR-10	Support agent views order, shipment and return/refund status without payment data.	Support	Must	API response for support role contains no payment amount/gateway fields in 100% of tests.
FR-11	Customer or support agent requests a return/refund for a paid order.	Customer, Support	Must	Refund created in Requested only for a paid order owned by the requester.
FR-12	Finance reviewer approves or rejects refunds with a mandatory reason on rejection; high-value refunds need a second approver.	Finance	Must	Customer/support approval attempts denied and audited; refund above limit stays pending without second approval.
FR-13	Approved refunds execute against the gateway with retry; failure never marks the refund Executed.	Finance	Must	Gateway failure → RetryPending; retry success → Executed once.
FR-14	Administrator assigns roles; store manager views consolidated sales, stock and refund reports.	Admin, Manager	Should	Role change audited; report totals equal sum of order records in 5 of 5 checks.

ID	Requirement	Source	Priority	Acceptance criteria
FR-15	Reservations expire if payment is not confirmed (added after validation S7).	Customer, Picker	Must	Unpaid reservation is released after 15 min and next buyer can purchase.

2.4 Non-functional requirements (measurable)

The vague statement “checkout should be fast” is replaced by NFR-01 below, which gives the operating condition and an observable threshold.

ID	Requirement	Source	Priority	Acceptance criteria
NFR-01	Performance: with 50 simultaneous users and a catalogue of 1,000 synthetic products, the 95th-percentile response time of place-order is ≤ 2 s.	Customer	Must	Load test script reports p95 ≤ 2 s over a 10-minute run.
NFR-02	Performance: payment callbacks are processed in ≤ 5 s (p95) at 100 events/min.	Finance	Must	Replay of 100 callbacks/min shows p95 ≤ 5 s.
NFR-03	Availability: order tracking and order placement are available ≥ 99.5% per month; a courier or gateway outage must not stop order placement.	Manager	Must	Stub outage test: orders still accepted; monthly availability computed from monitoring.
NFR-04	Security: server-side authorization on every endpoint; 0 unauthorised actions in a 40-case role matrix; no card data stored.	Admin, Finance	Must	Role matrix passes 40/40; database scan finds no card numbers.
NFR-05	Auditability: 100% of refund and role changes record user, time, old and new value; records cannot be edited via the UI.	Finance	Must	Audit sample check of 20 changes: 20/20 complete.
NFR-06	Usability: ≥ 8 of 10 first-time test users complete checkout in ≤ 3 minutes and ≤ 5 steps; WCAG 2.1 AA-oriented checks (contrast ≥ 4.5:1, keyboard operable, status not by colour alone).	Customer	Should	Usability session and accessibility checklist.
NFR-07	Recoverability: no confirmed order lost after restart; RPO ≤ 15 min, RTO ≤ 1 h (ASSUMPTION).	Manager	Must	Restart test; backup restore drill.
NFR-08	Integration resilience: failed external calls are stored locally and retried with back-off 1, 2, 4, 8, 16 min (max 5), then flagged for manual review.	Support	Must	Stub failure test confirms schedule and flag.

2.5 Prioritization method

I used **MoSCoW** to decide what is in Release 1 and **WSJF-lite** (value + risk reduction, divided by size; relative scores 1–10, ASSUMPTION) to order the Must items. Dependencies override scores, so rules that other features rely on (FR-03, FR-04, FR-05, NFR-04) come first.

Item	Value	Risk/Time	Size	WSJF	Comment
FR-04	10	10	5	4.0	Duplicate payments are the highest business and trust risk.
FR-03	10	9	5	3.8	Overselling the last unit.
FR-12/13	9	9	8	2.3	Refund control; depends on FR-11.
FR-09	7	8	5	3.0	Courier failure after the order is stored.
FR-14	5	3	5	1.6	Should – reports can come from simple summaries first.

2.6 How stakeholder conflicts shaped the specification

- **C1 (speed vs exact stock)** produced FR-03, FR-04 and later FR-15: reservation at order time with asynchronous payment confirmation and expiry, plus NFR-01 and NFR-02 thresholds instead of "fast".
- **C2 (support visibility vs privacy)** produced FR-10 and NFR-04: the restriction is a server-side acceptance criterion, not a screen design choice.
- **C3 (refund speed vs control)** produced FR-12/FR-13 and NFR-05, and removed approval rights from the support role.

2.7 Assumptions and unresolved questions

ID	Assumption / open question
A1	Gateway sends a unique event ID with each callback and signs it (ASSUMPTION; to be confirmed with gateway documentation).
A2	Single tax rate in Release 1; regional rules arrive in Q6.
A3	Courier API supports an idempotency key or lookup by order ID (ASSUMPTION).
OQ1	Should low-value refunds (e.g. below ₹500) be auto-approved? Left as a finance decision; Release 1 requires approval for all.
OQ2	Is a 15-minute reservation acceptable to the business for slow payment methods (e.g. bank transfer)?
OQ3	Which return window applies (7 or 10 days)? Treated as configuration.

Q3 – UML Models and Traceability

All diagrams are drawn from PlantUML-style text; the use case, class and checkout sequence diagrams are my original evidence screenshots. The state machines, component/deployment diagrams and the callback/courier-retry sequence were generated for this report (Graphviz and a plotting script) from the same model.

3.1 Use case diagram (original evidence)

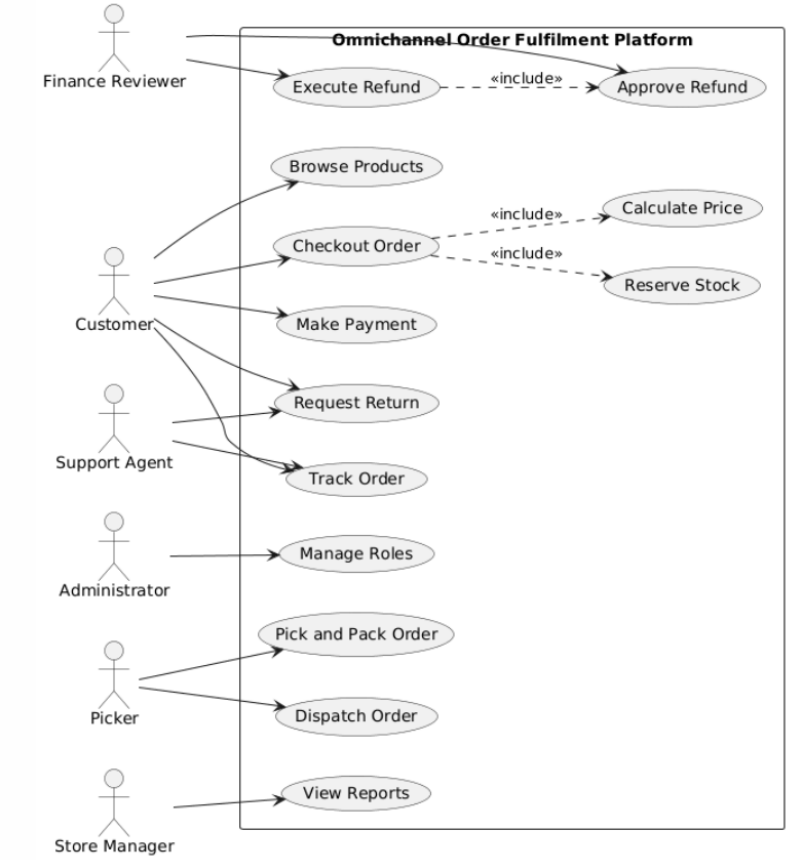


Figure 1 – Use case diagram: Omnichannel Order Fulfilment Platform (original evidence). Covers FR-01, 02, 03, 06, 07, 08, 10–14.

Actors: Customer, Support Agent, Finance Reviewer, Picker, Store Manager, Administrator. Checkout Order «include» Calculate Price and Reserve Stock; Execute Refund «include» Approve Refund, which ensures an unapproved refund cannot be executed.

3.2 Class diagram (original evidence)

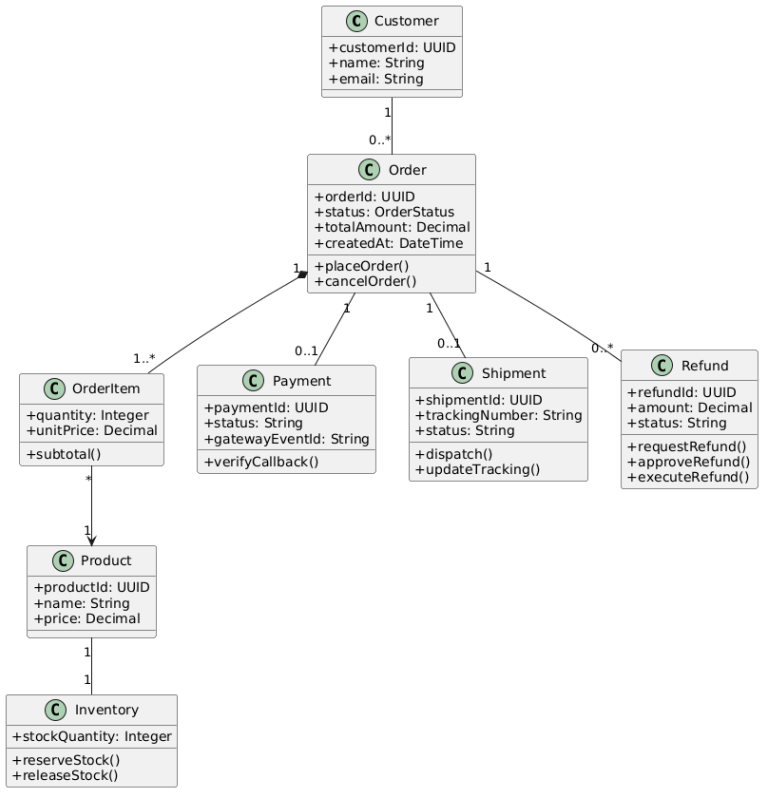


Figure 2 – Class diagram of the domain (original evidence).

Order composes OrderItems; Order has 0..1 Payment, 0..1 Shipment and 0..* Refunds; Product has one Inventory. Payment holds gatewayEventId (unique key for duplicate callbacks). Refund has requestRefund(), approveRefund(), executeRefund() operations separated so approval cannot be skipped.

3.3 Sequence diagrams

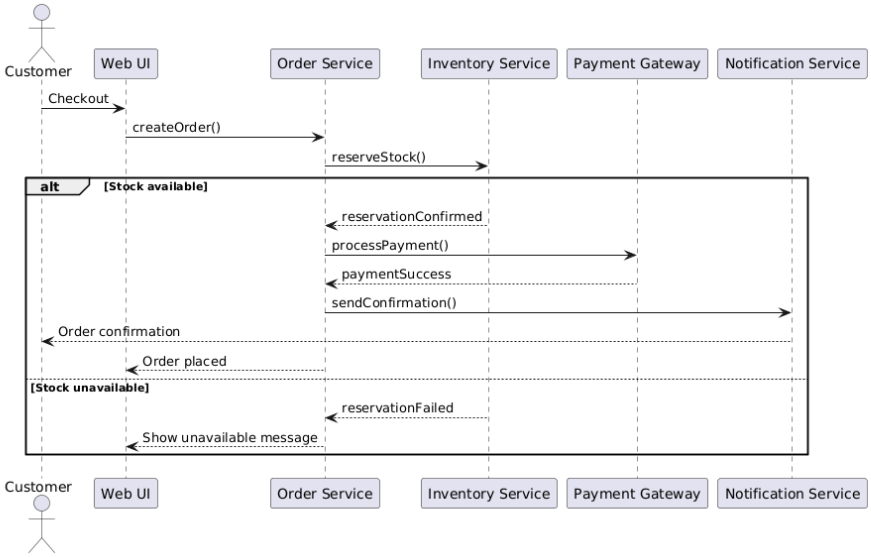


Figure 3 – Sequence diagram: checkout, stock reservation, payment and confirmation, with stock-unavailable alternative (original evidence).

Figure 3 covers FR-02, FR-03 and the stock-unavailable path. Figure 4 below traces the **payment callback → order confirmation → shipment creation → courier synchronization failure and retry** required by the assignment.

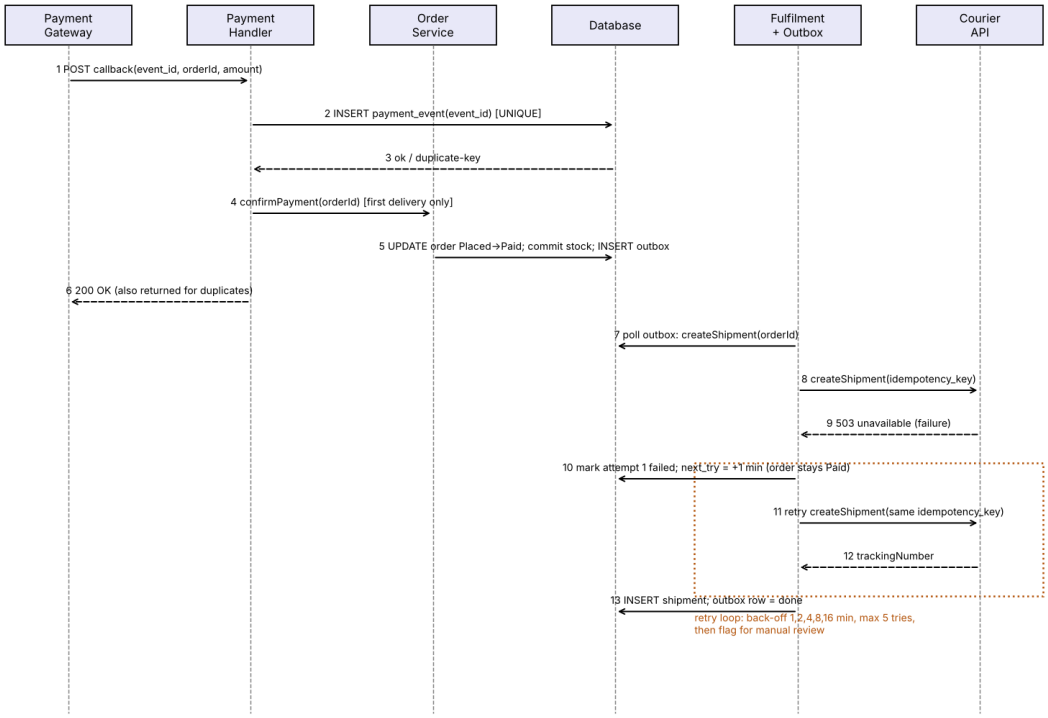


Figure 4 – Sequence diagram: payment callback, order confirmation, courier failure and retry (FR-04, FR-09, NFR-08).

3.4 Order state machine

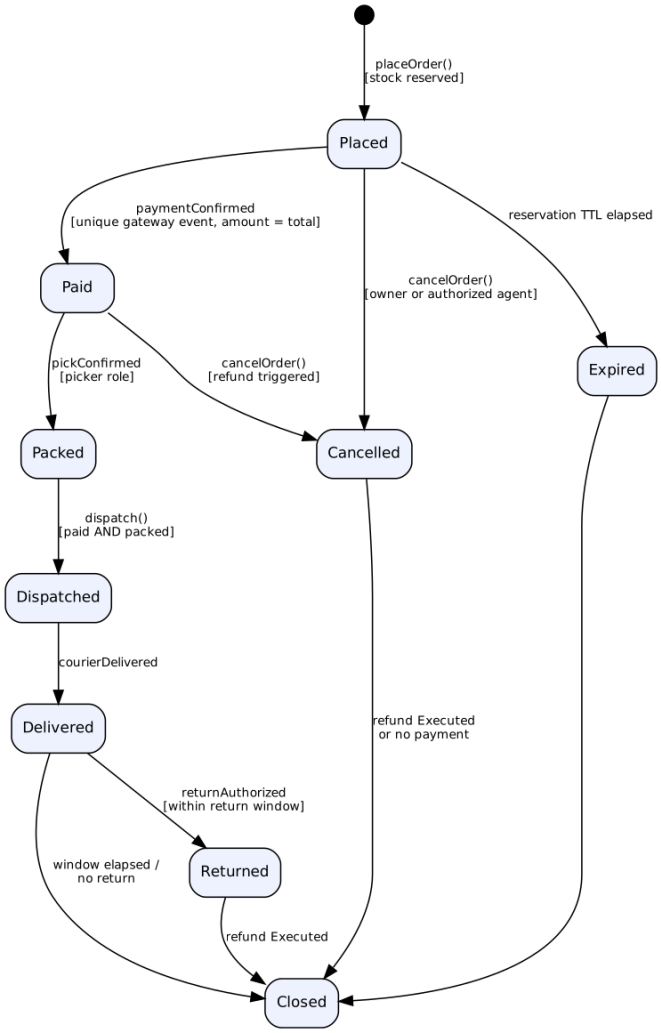


Figure 5 – Order state machine (Placed, Paid, Packed, Dispatched, Delivered, Closed, with Cancelled, Expired, Returned).

3.5 Refund behaviour: approval, rejection, execution, rollback

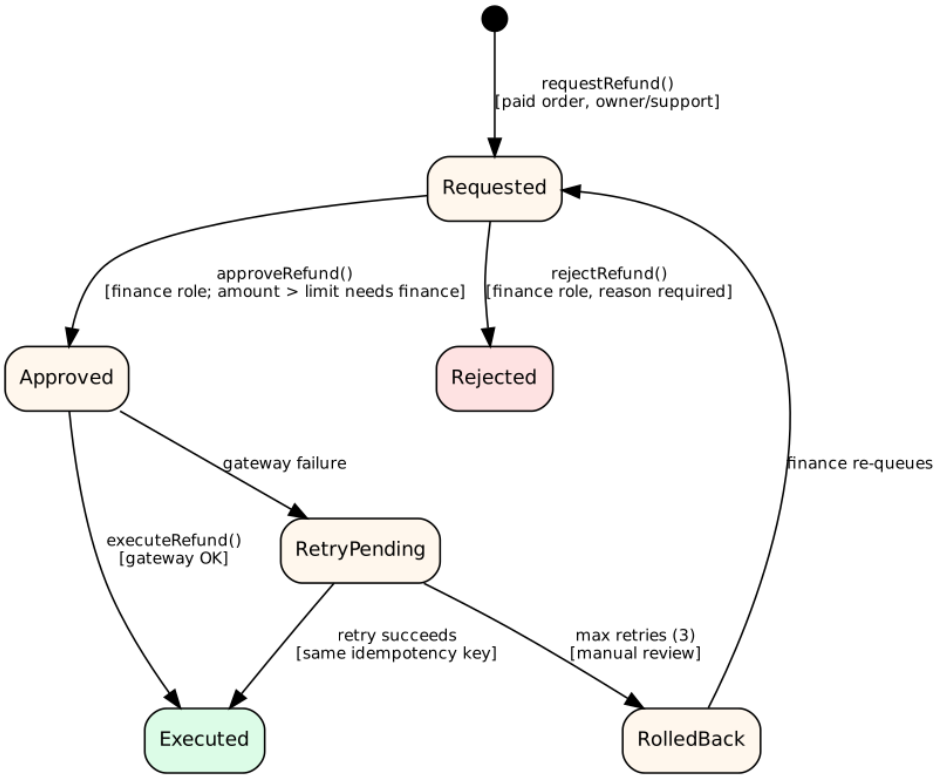


Figure 6 – Refund state machine (FR-11, FR-12, FR-13, NFR-05).

Approval and rejection are only reachable from Requested by the finance role. Execution is only reachable from Approved or RetryPending. If the gateway fails the refund stays RetryPending (never Executed); after 3 failed retries it is rolled back to a manual-review state and finance may re-queue it.

3.6 Inconsistencies found while connecting the models and corrections

#	Inconsistency found	Where	Correction
I1	Use case diagram has no use cases or actors for **payment callback handling**, **shipment creation/courier sync**, **cancel order**; the sequence diagrams involve Payment Gateway and Courier.	Use case vs sequence (Figs 1, 3, 4)	Add actors Payment Gateway and Courier API and use cases Handle Payment Callback, Create Shipment, Cancel Order, Approve/Reject Refund linked to FR-04, FR-09, FR-05.
I2	Sequence diagram (Fig 3) shows a synchronous “paymentSuccess” reply and no failure/duplicate branch, but the brief says callbacks are asynchronous and may be repeated; stock stays reserved if payment never arrives.	Sequence vs requirements	Callback modelled separately (Fig 4) with a unique event insert; added a payment-failed/expiry branch (FR-15).
I3	Class diagram has Inventory.stockQuantity but no reserved quantity or **Reservation** entity, yet FR-03 needs atomic reservation and state Expired.	Class vs state machine	Add Reservation (sku, qty, order, expiresAt) and Inventory.reservedQuantity; add Expired state.
I4	Class diagram has no **User/Role** or **AuditEntry**, and Refund lacks approvedBy, reason and attempts, but NFR-04/05 need role checks and an audit trail.	Class vs NFRs	Add User(role), AuditEntry, Refund.approvedBy/reason/retryCount.

#	Inconsistency found	Where	Correction
I5	Use case “Request Return” exists but class diagram has no ReturnRequest and the state machine had no Returned state.	Use case vs class vs state	Add ReturnRequest linked to Order; Returned state added to Fig 5.

3.7 Traceability matrix (Requirement → UML element → Component → Validation scenario)

Requirement	UML element	Architectural component	Validation scenario
FR-02	Use case Calculate Price; class Order.totalAmount	Order & Checkout	Unit tests (20 price/tax cases)
FR-03	Use case Reserve Stock; Inventory.reserveStock(); Fig 3	Inventory & Reservation	S1 last unit
FR-04	Payment.gatewayEventId; Fig 4 steps 1–6	Payment Integration + DB unique key	S2 repeated callback; S8 burst
FR-05	Order state machine (Fig 5)	Order & Checkout	S4 dispatch guard
FR-07/08	Use cases Pick and Pack, Dispatch Order	Fulfilment	S4
FR-09	Fig 4 steps 7–13; Shipment.dispatch()	Outbox & Retry Worker	S5 courier retry
FR-10	Use case Track Order; support role	Auth & RBAC + Order API	S6 role abuse
FR-12	Use cases Approve/Execute Refund; Refund state machine	Refund & Approval, Audit Log	S6; S10 queued approval
FR-13	Refund.executeRefund(); Fig 6	Refund & Approval + Payment Integration	S3 refund failure/retry
FR-15	Reservation entity; Expired state	Inventory & Reservation, Worker	S7 abandoned checkout
NFR-01/02	Fig 3, Fig 4	API + DB	S8 burst (model); load test proposed
NFR-04	User/Role class	Auth & RBAC	S6; 40-case role matrix
NFR-08	Fig 4 retry loop	Outbox & Retry Worker	S5

Q4 – Alternative Architectures and Justified Design

4.1 Alternative A – Modular monolith (one deployable, one PostgreSQL)

- **Components:** Auth & RBAC, Catalogue, Order & Checkout, Inventory & Reservation, Payment Integration, Fulfilment, Refund & Approval, Reporting, Outbox & Retry Worker, Audit Log – separate modules with clear interfaces inside one FastAPI application.
- **Communication / data ownership:** in-process calls; each module owns its tables (schema per module) and others use its interface, not its tables. One database transaction can cover order, stock and outbox.
- **Duplicate payment prevention:** UNIQUE constraint on gateway_event_id inserted first in the same transaction; a duplicate violates the key and returns 200 with no side effect.
- **Overselling prevention:** conditional atomic update (reserve only if stock − reserved ≥ qty) with row-level lock; rows updated = 0 means refused.

4.2 Alternative B – Event-driven microservices

- **Components:** Order, Inventory, Payment, Fulfilment, Refund, Notification and Reporting services plus API gateway and message broker.
- **Communication / data ownership:** asynchronous events (OrderPlaced, StockReserved, PaymentConfirmed…); each service has its own database; consistency by saga with compensations.
- **Duplicate payment prevention:** idempotent consumer with processed-event table in the Payment service; broker may redeliver, so every consumer must deduplicate.
- **Overselling prevention:** Inventory service is the single owner of stock and serializes reservation commands per SKU; Order waits for StockReserved or StockRejected.

4.3 Comparison

Attribute	A – Modular monolith	B – Event-driven microservices
Scalability	Scale by running more identical instances; DB is the shared limit. Enough for 50→500 users and 100→1,000 events/min (ASSUMPTION).	Scales services independently; best for very large/variable loads.
Consistency	Strong (single ACID transaction for order + stock + outbox).	Eventual; needs saga and compensations; harder to guarantee refund invariants.
Availability	One deployable is a single point of failure; mitigated by replicas and backups.	Failure isolation per service, but more moving parts that can fail (broker, network).
Security	One place to enforce RBAC and audit.	Auth and audit must be consistent across services; larger attack surface.
Operational complexity / cost	Low: one app, one DB; fits a small team and limited budget.	High: broker, many deployments, distributed tracing; needs more staff and money.
Likely failure points	Database, long transactions, module coupling.	Message loss/duplication, saga stuck states, version mismatches.
Maintenance	Single codebase; risk of tangling unless module boundaries are enforced.	Independent releases but contract management overhead.

4.4 Selection

Selected: Alternative A – modular monolith with a transactional outbox. The organization has a small technical team and a modest deployment, and the key rules (no duplicate payment, no overselling, no unapproved refund) need strong consistency, which one database transaction gives cheaply. Microservices would add a broker, sagas and more deployments without a demonstrated scale need. Module boundaries and the outbox keep the option to extract a service later (for example Notification or Reporting) if sustained load requires it.

4.5 Component diagram (selected architecture)

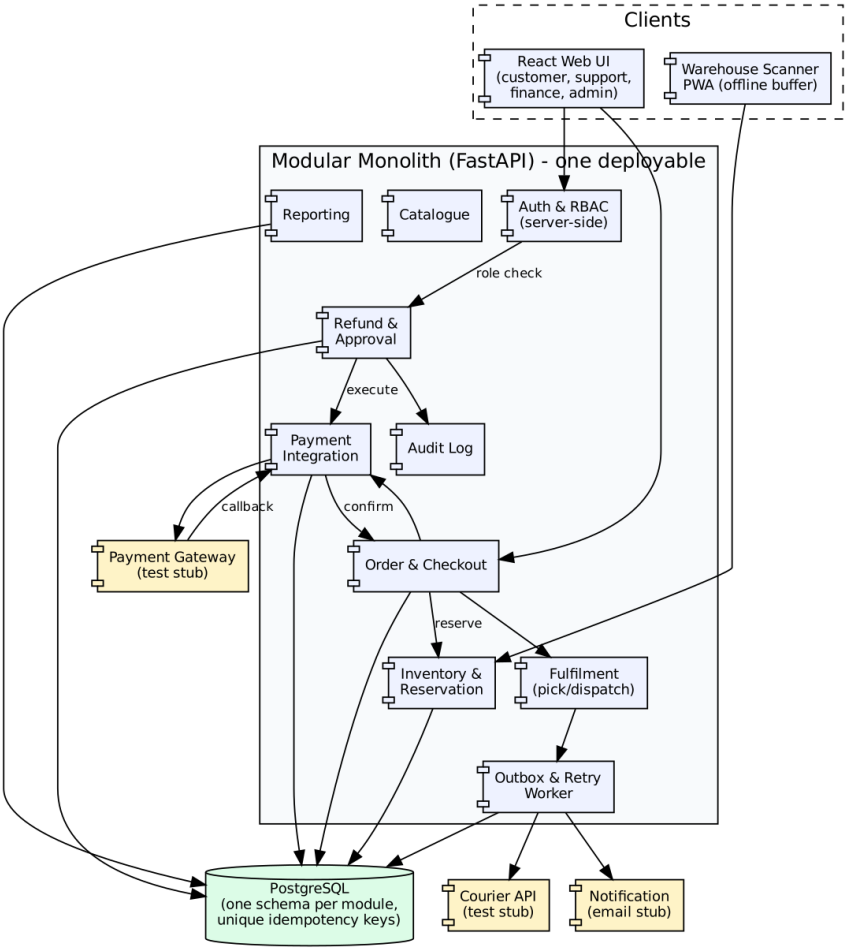


Figure 7 – Component diagram: modular monolith with PostgreSQL and external test stubs.

4.6 Deployment diagram (selected architecture, Release 1)

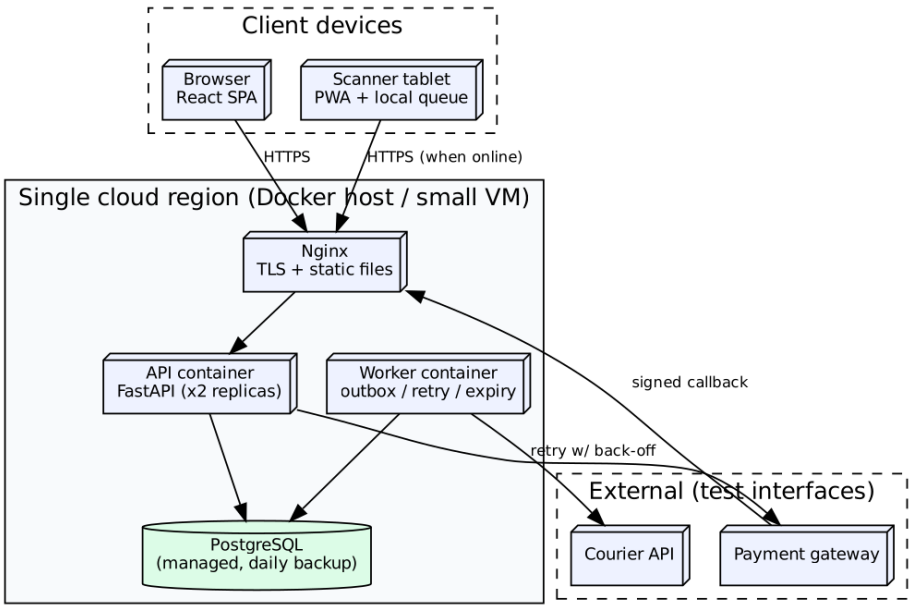


Figure 8 – Deployment diagram: Nginx, two API replicas, a worker and PostgreSQL in one region, with Docker containers (reference choice, not compulsory).

4.7 Architectural decision records

ADR-001: Architecture style	
Problem	Need consistent order state with a small team and modest budget.
Alternatives	(1) Modular monolith; (2) event-driven microservices.
Selection	Modular monolith.
Justification	Single ACID transaction covers order, stock and outbox; lowest cost and complexity.
Consequences	Scaling limited by the DB; module discipline required; service extraction later needs planning.

ADR-002: Idempotency by unique key + transactional outbox	
Problem	Duplicate callbacks and failed courier sync after order is stored.
Alternatives	(1) In-memory de-duplication; (2) DB unique key + outbox table; (3) broker de-duplication.
Selection	DB unique key on gateway_event_id and an outbox with idempotency keys for external calls.
Justification	Survives restarts and concurrent deliveries; external call failures never roll back the stored order.
Consequences	Outbox polling adds small delay (seconds); extra table maintenance.

ADR-003: Atomic reservation with expiry	
Problem	Two buyers for the last unit; abandoned payments must not block stock.
Alternatives	(1) Check then update; (2) lock until paid; (3) conditional atomic reservation with TTL.
Selection	Conditional atomic update plus 15-minute expiry worker (expiry added after validation S7).
Justification	Prevents overselling and releases unpaid stock.
Consequences	Slow payments may lose their reservation (open question OQ2).

ADR-004: Server-side RBAC and gated refund state machine	
Problem	Support agent must not gain finance privileges by choosing a role in the UI.
Alternatives	(1) UI-level role checks; (2) server-side permissions on each endpoint plus state-machine guard.
Selection	Server-side RBAC; approval and execution as separate guarded transitions.
Justification	Role claims come from the authenticated session, not the client; audit on every denial.
Consequences	Every endpoint needs a permission test (role matrix upkeep).

Q5 – Validate Critical Behaviour and Revise the Design

5.1 Invariants

ID	Invariant (must hold at all times)
INV-1	A gateway event ID produces at most one payment and one order confirmation.
INV-2	Reserved + sold quantity of a product never exceeds its stock (no overselling).
INV-3	A refund can be Executed only if it was Approved by a finance reviewer; a failed gateway call never leaves a refund in Executed.
INV-4	An order is Dispatched only if payment is confirmed and picking is confirmed.
INV-5	A customer can read only their own orders; a support role never receives payment fields.
INV-6	Every refund approval, rejection, execution and denied attempt is written to the audit log.
INV-7	An order total never changes after placement.
INV-8	A buffered scan or stock event with the same event ID is applied once.

5.2 Operation contracts

Operation	Preconditions	Postconditions
processPaymentCallback(eventId, orderId, amount)	Signature valid (ASSUMPTION); order is Placed; amount equals order total; eventId not seen before.	One payment stored; order Paid; reservation converted to sold stock; one outbox row created. If eventId seen: no state change, 200 returned.
executeRefund(refundId)	Refund status is Approved or RetryPending; approver has finance role.	Gateway success → Executed and audit entry. Gateway failure → RetryPending, retry count +1, never Executed.
reserveStock(sku, qty)	qty > 0; stock − reserved ≥ qty.	reserved increases by qty; otherwise unchanged and the caller is refused.

5.3 Validation method

I wrote a small **executable model** in Python (about 130 lines, synthetic data, in-memory, thread locks standing in for database row locks and unique keys) and ran ten scenarios. Predictions were written first (5.4); observed output is in Appendix B. The model demonstrates that the rules are consistent and enforceable as designed; it does **not** test PostgreSQL, a real gateway, signature verification, or performance.

5.4 Validation scenarios: predicted vs observed

#	Scenario	Predicted behaviour (before running)	Enforced by	Observed (DEMONSTRATED)
S1	Two customers buy the last unit concurrently	One order created, one refused	FR-03, ADR-003	1 created, 1 refused – matched
S2	Same payment callback arrives 5× at the same time	One payment, one confirmation, stock reduced once	FR-04, ADR-002	paid = 1, duplicates = 4, payments = 1, stock 5→4 – matched
S3	Approved refund fails at gateway	Unapproved execute rejected; failure → RetryPending; retry → Executed once	FR-12, FR-13	Rejected before approval; first execute retry-pending; retry executed – matched

#	Scenario	Predicted behaviour (before running)	Enforced by	Observed (DEMONSTRATED)
S4	Dispatch without payment or pick	Rejected until paid and packed	FR-08	rejected; rejected; dispatched after pay + pack – matched
S5	Courier fails twice then recovers	Order kept; shipment created on third try	FR-09, NFR-08	pending-retry ×2 then synced; order status unchanged – matched
S6	Customer/support tries to approve refund; other customer reads order	All denied and audited; support view has no payment fields	FR-12, NFR-04, ADR-004	All denied; refund stayed Requested; no payment field – matched
S7	Customer abandons payment while holding last unit	Reservation should expire; next buyer succeeds	FR-15 (not yet written)	**Model v1: FAILED** – second buyer blocked forever. Model v2 (with TTL): passed

5.5 Weakness found and design revision

Weakness: scenario S7 failed in the first model. The first design reserved stock at order time but had no way to release it if payment never arrived, so an abandoned checkout could block the last unit indefinitely (stock-denial). This was also a gap in the original sequence diagram (no payment-failed branch).

Revision: (1) new requirement FR-15 and open question OQ2; (2) Reservation entity and Expired state added to the class diagram and Fig 5; (3) ADR-003 updated with a 15-minute expiry worker; (4) payment-failed/expiry branch added to the checkout sequence; (5) model v2 re-run: all ten scenarios pass (Appendix B). Remaining limits: expiry is only demonstrated with a 1-second test TTL; a customer paying just after expiry needs a defined rule (proposed: reject payment, auto-refund; not demonstrated).

Q6 – Redesign for Expansion and Partial Failure

Changed requirement: from three to twelve regions; from 50 to 500 simultaneous users; from 100 to 1,000 stock, payment and scan events per minute (design assumptions, not demonstrated capacity); regional tax/shipping/refund-approval rules; offline scanner buffering; locally queued approvals not shown as globally approved; consolidated reports; limited budget increase.

A. Diagnose the impact

Area	Affected items	Impact	Quality attribute
Requirements	FR-02 (single tax rate), FR-12 (single approval limit), FR-14 (reporting), NFR-01/02/03	Tax, shipping and approval limit become per-region data; thresholds must hold at 500 users; reports span 12 regions.	Modifiability, performance
Assumptions	A2 (single tax); 100 events/min; 50 users; always-online scanners	Event rate ×10 (≈17 events/s average; peak assumed 5× ≈ 83/s); scanners can be offline.	Availability, consistency
Components	Inventory, Payment, Refund, Reporting, Worker	Need regional rule lookup, batch ingest with de-duplication, pending-approval display, read-optimised reporting.	Scalability, correctness
Data	Stock, events, approvals	Duplicate scan batches possible after reconnect; local approval ≠ global approval.	Consistency
Budget	Infrastructure cost	Only limited increase allowed.	Cost

B. Revised architecture

Decision: keep the modular monolith and **scale it up, not apart**: more stateless API replicas, a PostgreSQL read replica for reports, regional rule tables and a batch ingest endpoint. No broker or per-region deployment is added, because the planning load (≈17 events/s average) does not justify it and the budget is limited. Each change is tied to a requirement below.

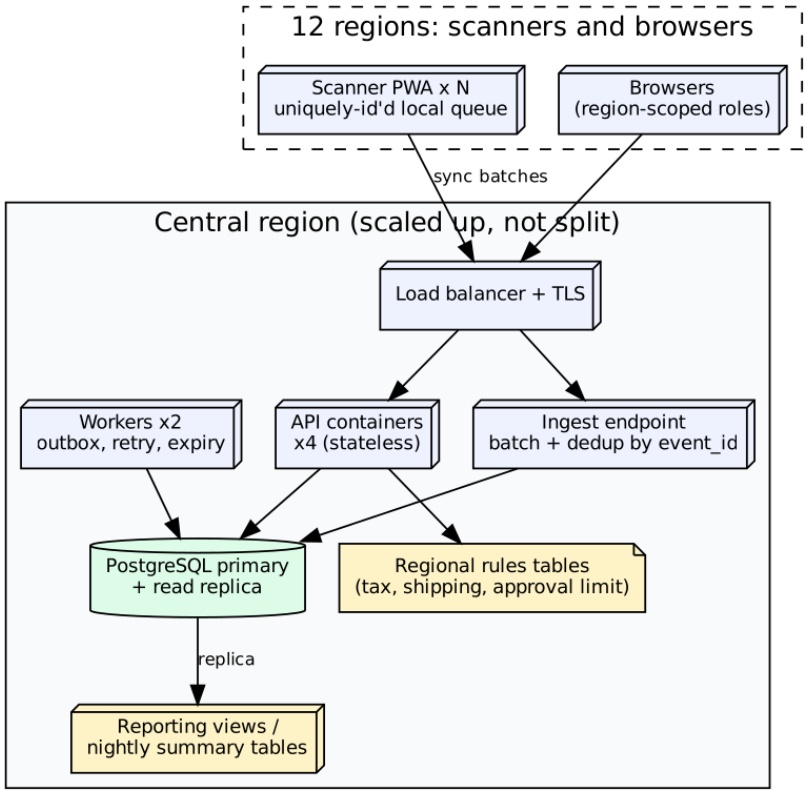


Figure 9 – Revised deployment diagram for 12 regions (central region scaled up; scanners buffer locally).

Change	What changes	Linked requirement / limitation
Regional rules table	Tax, shipping and refund-approval limit stored per region and versioned; order stores the rule version used.	Each region defines local rules; INV-7 (total fixed).
API replicas 2 → 4 (ASSUMPTION)	Stateless containers behind a load balancer.	500 simultaneous users, NFR-01.
Batch ingest endpoint	Scanners send batches of events with client-generated unique IDs; server de-duplicates per ID.	1,000 events/min; offline scanner; INV-8.
Read replica + summary tables	Reports run on replica; summaries refreshed every 5 minutes.	Consolidated reports without slowing checkout.
Pending-approval status	Locally queued approval shows “Pending server validation”; becomes Approved only after the server verifies role, region and limit.	INV-3; no global approval before validation.

Synchronization and deduplication rules

Rule	Definition
SR1	Every scan, stock and payment event carries a UUID created when the event occurs (on the device for scans).
SR2	The scanner stores events in an ordered local queue and sends them in batches when online; unsent events are never discarded.
SR3	The server records processed event IDs (unique constraint); a repeated ID is acknowledged but not applied again.
SR4	If a batch partly fails, only the unacknowledged events are resent; the server returns per-event results.

Rule	Definition
SR5	The server is the authority for stock: if applying a buffered decrement would make stock negative, the event is flagged for human stock correction, not silently adjusted.
SR6	A queued refund approval is a request: the UI shows “Pending server validation”; the server checks the approver's role, region and limit and then sets Approved or returns Rejected. Execution is impossible before that.

C. Validate the redesign

#	Scenario	Predicted	Observed (DEMONSTRATED in executable model)
S8	Increased load: 1,000 callbacks (500 orders, each sent twice, 8 workers)	500 paid, 500 payments, stock reduced by exactly 500	paid = 500, payments = 500, stock 2000→1500. In-memory run time is not a capacity claim.
S9	Offline scanner buffer replayed after reconnect (duplicates)	Each scan ID applied once; final stock 11	First sync applied 3; replay applied 1 new and ignored 3 duplicates; stock = 11.
S10	Partial failure / queued approval before server validation	Refund stays Requested and cannot be executed until the server approves	Server status Requested; execute before sync rejected; after sync Approved.

Limitations of this evidence: single machine, in-memory, no network failure, no real PostgreSQL. A realistic load test (e.g. a script at 83 events/s peak against the real stack) and a network-partition test are **proposed**, not done.

D. Defend the design

Aspect	Explanation
Benefits	Keeps strong consistency for orders and refunds; handles the planned load with small changes; cost rises only by extra replicas and one read replica (indicative 1.5–2× compute cost, ASSUMPTION); regional flexibility through data, not code forks.
Consistency vs availability	Scanners favour availability (work offline) but their effect on stock is applied only after server validation (eventual consistency for scans). Approvals, refunds and payments favour consistency (server decides). Reports may be up to 5 minutes old.
Cost trade-off	No broker, no per-region stacks: cheaper to run and to staff; but the central region is a single point of failure and far regions see higher latency.
Why not more services	Adding services does not by itself increase scalability; the identified bottleneck is read load and burst events, solved by replicas, batching and summaries.
Remaining limitations	Capacity is assumed, not measured; one central DB may need partitioning or a service split if sustained load exceeds the assumption; regional legal rules need review; dual-approval tier is specified but not demonstrated in the model.

Q7 – Individual Technical Reflection

Requirement with the greatest influence. FR-04 (one payment and one confirmation per gateway event) shaped the architecture most: it led to the unique-key rule, the transactional outbox and the choice of a single-database modular monolith, because strong consistency was cheaper than distributed de-duplication (ADR-001, ADR-002).

Most difficult conflict. C3, refund speed versus finance control. Support and customers want quick refunds, but the control cannot be bypassed. I resolved it with a queue, a response-time target and a second approver for high values, but the right limit and any auto-approval for low-value refunds (OQ1) remain a business decision.

Inconsistency discovered. Connecting the diagrams exposed that the class diagram had no Reservation, and S7 then showed the cost: an abandoned payment could hold the last unit forever. The use case diagram also lacked the gateway and courier actors (I1, I3, S7).

Trade-off still significant. A single central region keeps consistency and cost low but adds latency and a single point of failure for twelve regions; the alternative (regional deployments) would add the consistency problems I avoided.

Evidence still needed before implementation. Real gateway and courier documentation (event IDs, signatures, idempotency keys); a load test against real PostgreSQL at the assumed peak; a network-partition test of the scanners; finance decisions on approval limits and auto-approval; security review of RBAC and audit; usability testing with real users; and the Jira/Confluence records captured from the live project.

Sources and Tools Acknowledgement

- Martin Fowler, Patterns of Enterprise Application Architecture (idempotency, outbox, modular monolith discussions); Sam Newman, Building Microservices; Ian Sommerville, Software Engineering.
- UML 2.x notation; PlantUML (use case, class and checkout sequence diagrams – my original evidence); Graphviz (state machine, component and deployment diagrams); Python/Matplotlib (callback sequence diagram); Python (executable model).
- Atlassian Jira and Confluence documentation; GitHub and Azure DevOps documentation (verify current features and prices).
- An AI assistant (Claude) helped draft and structure this report and write the executable model. All stakeholder responses, thresholds and workloads are simulated assumptions; I am responsible for reviewing and customising the content.

Appendix A – Evidence Checklist and Capture Status

Ref.	Evidence	Status
A1	Use case diagram	Included (Figure 1, original screenshot).
A2	Class diagram	Included (Figure 2, original screenshot).
A3	Checkout sequence diagram	Included (Figure 3, original screenshot).
A4	Callback / courier-retry sequence, order and refund state machines, component and deployment diagrams	Included (Figures 4–9), generated from the same model; not PlantUML screenshots.
A5	Jira project OFP with FR/NFR issues, priorities, links	Not captured in the supplied evidence – to be attached by the student.
A6	Confluence specification and ADR pages	Not captured – to be attached by the student.
A7	Executable model source and run output	Appendix B; files model.py and run.py supplied separately.

Evidence limitation: Jira/Confluence records are described as a planned structure and are not claimed as completed until screenshots are attached. Validation results are from an in-memory model.

Appendix B – Executable Model Run Output (observed)

Model v1 (no reservation expiry) – summary line

SUMMARY v1 {'S1': 'PASS', 'S2': 'PASS', 'S3': 'PASS', 'S4': 'PASS', 'S5': 'PASS', 'S6': 'PASS', 'S7': 'FAIL', 'S8': 'PASS', 'S9': 'PASS', 'S10': 'PASS'}

S7 observed in v1: second buyer blocked forever by unpaid reservation; first order status = Placed → FAIL.

Model v2 (with 15-minute expiry; 1-second TTL used in the test) – full output

```
S1 Two customers, last unit (concurrent)
  expected : exactly 1 order created, 1 refused
  observed : 1 created, 1 refused
  result   : PASS
S2 Same gateway event delivered 5x concurrently
  expected : 1 payment, 1 confirmation, stock 5->4
  observed : paid=1, duplicates=4, payments=1, stock=4, confirmations=1
  result   : PASS
S3 Refund gate + gateway failure + retry
  expected : unapproved execute rejected; failure -> RetryPending (not Executed); retry -> Executed once
  observed : unapproved=rejected-not-approved; approve=approved; 1st execute=retry-pending (status
RetryPending); retry=executed (status Executed)
  result   : PASS
S4 Dispatch guard
  expected : rejected unpaid; rejected paid-but-unpacked; allowed after pay+pack
  observed : rejected; rejected; dispatched
  result   : PASS
S5 Courier outage then retry
  expected : order record kept; 2 failures then synced; no loss of order
  observed : pending-retry, pending-retry, synced, synced; order status=Dispatched; tracking=TRK-af65f1
  result   : PASS
S6 Role abuse
  expected : customer/support approve denied+audited; cross-customer view denied; support view hides
payment amount
  observed : cust approve=denied; support approve=denied; other-customer view=denied; support view has
payment field=False; refund status=Requested
  result   : PASS
S7 Abandoned checkout (no payment) holds last unit
```

  expected : reservation expires after TTL; second buyer succeeds
  observed : second buyer got the unit; first order status=Expired
  result   : PASS
S8 Burst: 1,000 callbacks (500 unique orders, each sent twice, 8 workers)
  expected : 500 paid, 500 payments, stock 2000->1500
  observed : paid=500, payments=500, stock=1500, in-memory time=0.001s (not a capacity claim)
  result   : PASS
S9 Offline scanner buffer replayed twice
  expected : each scan id applied once; stock 10 -1 -1 +5 -2 = 11
  observed : first sync applied/dup=(3, 0); replay applied/dup=(1, 3); stock=11
  result   : PASS
S10 Locally queued approval
  expected : shown as 'Pending server validation'; server status stays Requested until sync; execute blocked before sync
  observed : local label='Pending server validation'; server before sync=Requested; execute before sync=rejected-not-approved; server after sync=Approved
  result   : PASS
SUMMARY v2 {'S1': 'PASS', 'S2': 'PASS', 'S3': 'PASS', 'S4': 'PASS', 'S5': 'PASS', 'S6': 'PASS', 'S7': 'PASS', 'S8': 'PASS', 'S9': 'PASS', 'S10': 'PASS'}